

Университет ИТМО

Факультет программной инженерии и компьютерной техники
Образовательная программа системное и прикладное
программное обеспечение

Лабораторная работа №4
По дисциплине "Базы данных"
Вариант 4003456900

Выполнил студент группы Р3109
Евграфов Артём Андреевич
Проверила:
Воронина Дарья Сергеевна

Санкт-Петербург 2025

Содержание

1. Задание варианта 4003456900	2
2. Реализация запросов на SQL	2
2.1. Запрос 1	2
2.2. Запрос 2	4
3. Вывод	6

1. Задание варианта 4003456900

Внимание! У разных вариантов разный текст задания!

Составить запросы на языке SQL (пункты 1-2).

Для каждого запроса предложить индексы, добавление которых уменьшит время выполнения запроса (указать таблицы/атрибуты, для которых нужно добавить индексы, написать тип индекса; объяснить, почему добавление индекса будет полезным для данного запроса).

Для запросов 1-2 необходимо составить возможные планы выполнения запросов. Планы составляются на основании предположения, что в таблицах отсутствуют индексы. Из составленных планов необходимо выбрать оптимальный и объяснить свой выбор. Изменяются ли планы при добавлении индекса и как?

Для запросов 1-2 необходимо добавить в отчет вывод команды EXPLAIN ANALYZE [запрос]

Подробные ответы на все вышеперечисленные вопросы должны присутствовать в отчете (планы выполнения запросов должны быть нарисованы, ответы на вопросы - представлены в текстовом виде).

1. Сделать запрос для получения атрибутов из указанных таблиц, применив фильтры по указанным условиям:

Таблицы: Н_ОЦЕНКИ, Н_ВЕДОМОСТИ.

Вывести атрибуты: Н_ОЦЕНКИ.КОД, Н_ВЕДОМОСТИ.ДАТА.

Фильтры (AND):

a) Н_ОЦЕНКИ.КОД > 2.

b) Н_ВЕДОМОСТИ.ДАТА < 1998-01-05.

c) Н_ВЕДОМОСТИ.ДАТА = 2022-06-08.

Вид соединения: RIGHT JOIN.

2. Сделать запрос для получения атрибутов из указанных таблиц, применив фильтры по указанным условиям:

Таблицы: Н_ЛЮДИ, Н_ВЕДОМОСТИ, Н_СЕССИЯ.

Вывести атрибуты: Н_ЛЮДИ.ОТЧЕСТВО, Н_ВЕДОМОСТИ.ИД, Н_СЕССИЯ.ИД.

Фильтры (AND):

a) Н_ЛЮДИ.ФАМИЛИЯ < Афанасьев.

b) Н_ВЕДОМОСТИ.ДАТА = 1998-01-05.

Вид соединения: LEFT JOIN.

2. Реализация запросов на SQL

2.1. Запрос 1

```
1 SELECT "Н_ОЦЕНКИ"."КОД", "Н_ВЕДОМОСТИ"."ДАТА"
2 FROM "Н_ОЦЕНКИ"
3     RIGHT JOIN "Н_ВЕДОМОСТИ" ON "Н_ВЕДОМОСТИ"."ОЦЕНКА" = "Н_ОЦЕНКИ"."КОД"
4 WHERE ("Н_ОЦЕНКИ"."КОД" ~ '^\\d+$' AND "Н_ОЦЕНКИ"."КОД"::integer > 2)
5     AND "Н_ВЕДОМОСТИ"."ДАТА" < '1998-01-05'::date
6     AND "Н_ВЕДОМОСТИ"."ДАТА" = '2022-06-08'::date;
```

В запросе присутствует очевидное противоречие (замыкание), поэтому планировщик запросов сразу его распознает и выдаст 0 строк. Давайте для индексов изменим запрос: пусть мы хотим даты > '1998-01-05' и < '2022-06-08' (остальное без изменений).

Индексы:

1. Для сущности Н_ВЕДОМОСТИ индекс на атрибуте ДАТА (B-tree, выборка происходит с использованием операторов сравнения, поэтому оптимально использование именно B-tree). Это ускорит фильтрацию строк с условием Н_ВЕДОМОСТИ.ДАТА > '1998-01-05' и Н_ВЕДОМОСТИ.ДАТА < '2022-06-08'.

CREATE INDEX ON "Н_ВЕДОМОСТИ" USING BTREE("ДАТА");

tablename	attname	n_distinct	null_frac	avg_width	correlation
Н_ВЕДОМОСТИ	ДАТА	1815	0	8	0.5304446

Уникальные значения: 1815,

NULL-элементов: 0,

Средний размер значений: 8 байт,

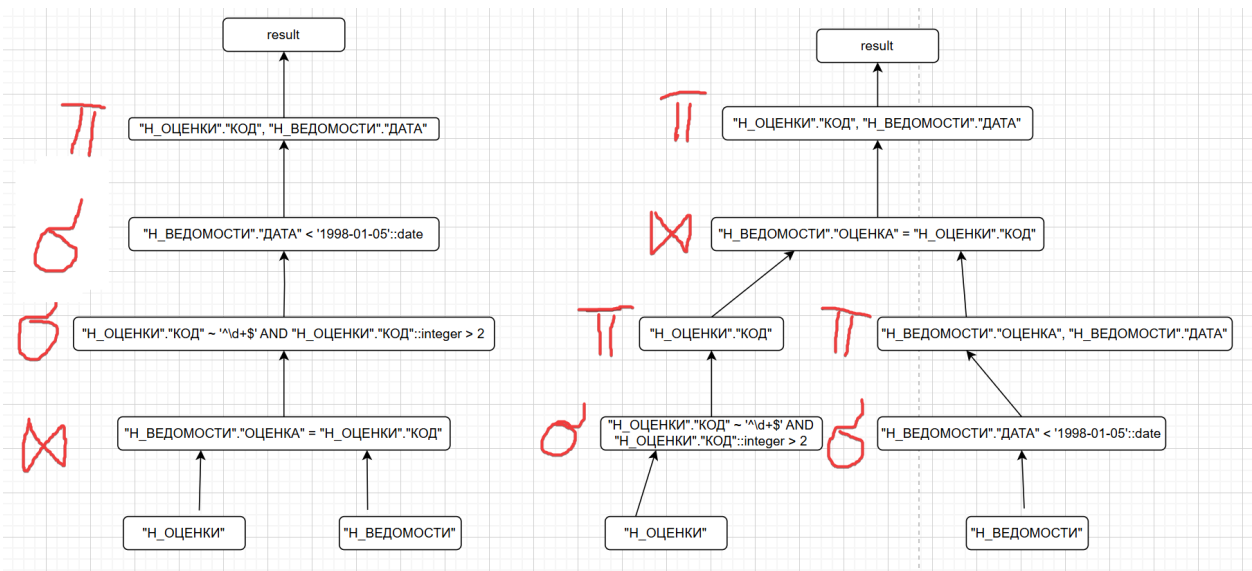
Всего элементов: 222440,

Уровень корреляции (корреляция между физическим порядком строк на диске и логическим порядком значений в этом столбце): 0.53 (чем выше корреляция, тем меньше случайных обращений к диску для чтения строк таблицы после нахождения их в индексе).

Все даты равномерно распределены (от 98-ого года до 22-ого), следовательно, запрос, который действует как раз в этом диапазоне, затронет как раз все 222440 строк. Для оптимизации этой затратной операции стоит использовать индекс, предложенный выше.

2. Индекс на "Н_ОЦЕНКИ"."КОД" не необходим, ибо у нас всего может быть 9 значений.

3. Аналогично для "Н_ВЕДОМОСТИ"."ОЦЕНКА" (множество значений совпадают).



Второй (правый) план является оптимальным, из-за того, что выборка происходит на более ранних этапах, идет соединение только нужных атрибутов, и, как следствие, размер промежуточных данных меньше. При добавлении индексов планы выполнения запросов изменятся:

Вместо полного сканирования будет использоваться индексный скан.

Nested Loops Join будет работать быстрее за счет индекса для ИД.

```

1 EXPLAIN ANALYSE SELECT "Н_ОЦЕНКИ"."КОД", "Н_ВЕДОМОСТИ"."ДАТА"
2   FROM "Н_ОЦЕНКИ"
3         RIGHT JOIN "Н_ВЕДОМОСТИ" ON "Н_ВЕДОМОСТИ"."ОЦЕНКА" =
4           ↳ "Н_ОЦЕНКИ"."КОД"
5   WHERE ("Н_ОЦЕНКИ"."КОД" ~ '^\\d+$' AND "Н_ОЦЕНКИ"."КОД"::integer > 2)
6         AND "Н_ВЕДОМОСТИ"."ДАТА" < '1998-01-05'::date
7         AND "Н_ВЕДОМОСТИ"."ДАТА" = '2022-06-08'::date;

```

QUERY PLAN	
1	Nested Loop (cost=0.29..8.38 rows=1 width=42) (actual time=0.030..0.031 rows=0 loops=1)
2	Join Filter: ((("Н_ОЦЕНКИ"."КОД")::text = ("Н_ВЕДОМОСТИ"."ОЦЕНКА")::text)
3	-> Seq Scan on "Н_ОЦЕНКИ" (cost=0.00..1.18 rows=1 width=34) (actual time=0.020..0.023 rows=4 loops=1)
4	Filter: (((("КОД")::text ~ '^\\d+\$'::text) AND ((("КОД")::integer > 2)))
5	Rows Removed by Filter: 5
6	-> Index Scan using "ВЕД_ДАТА_I" on "Н_ВЕДОМОСТИ" (cost=0.29..7.19 rows=1 width=14) (actual time=0.001..0.001 ro...
7	Index Cond: (("ДАТА" < '1998-01-05'::date) AND ("ДАТА" = '2022-06-08'::date))
8	Planning Time: 0.252 ms
9	Execution Time: 0.060 ms

Запрос выполняется с использованием Nested Loop для соединения, где одна таблица сканируется последовательно, а для другой используется индекс. Внешний цикл берет строки из "Н_ОЦЕНКИ" (с фильтрацией) и для каждой строки из цикла Postgre обращается к "Н_ВЕДОМОСТИ" и рассматривают строку, прошедшую фильтрацию с помощью индекса "ВЕД_ДАТА_I".

cost=0.29..8.38:

0.29 - оценочная стоимость запуска соединения

8.38 - оценочная общая стоимость выполнения всего соединения

rows=1 - оценочное планировщиком количество строк, которое вернет вся операция соединения

width=42 - средняя ширина (в байтах) одной итоговой строки после соединения.

actual time=0.030..0.031:

0.030 - фактическое время в ms до получения первой строки результата соединения

0.031 - фактическое общее время в ms на выполнение всей операции соединения

rows=0 - фактическое количество строк, которое было возвращено в результате соединения

loops=1: вся операция Nested Loop была выполнена 1 раз.

-> Index Scan using "ВЕД_ДАТА_I" on "Н_ВЕДОМОСТИ" - это операция доступа к данным таблицы "Н_ВЕДОМОСТИ" с использованием индекса "ВЕД_ДАТА_I".
0.29 - оценочная "стоимость запуска" (возврат первой строки)
7.19 - общая стоимость операции в условных единицах
rows=1 - оценочное количество строк, которые вернутся
widths - средняя ширина строк в байтах
actual time=0.001..0.001 - время в миллисекундах, чтобы получить первую строку из этого узла и общее время в миллисекундах, которое потребовалось для выполнения всей этой операции и возврата всех строк
rows=0 - фактическое количество строк, которое было возвращено этой операцией Index Scan во время выполнения
loops=4 - количество раз, которое этот узел был выполнен
Index Scan: Index Cond: ((ДАТА < '1998-01-05'::date) AND (ДАТА = '2022-06-08'::date)) - условие на индекс
Planning Time: 0.252 ms - время на анализ запроса
Execution Time: 0.060 ms - время на выполнение запроса

2.2. Запрос 2

```

1 SELECT "Н_ЛЮДИ"."ОТЧЕСТВО", "Н_ВЕДОМОСТИ"."ИД", "Н_СЕССИЯ"."ИД"
2 FROM "Н_ЛЮДИ"
3     LEFT JOIN "Н_ВЕДОМОСТИ" ON "Н_ВЕДОМОСТИ"."ЧЛВК_ИД" = "Н_ЛЮДИ"."ИД"
4     LEFT JOIN "Н_СЕССИЯ" ON "Н_СЕССИЯ"."ЧЛВК_ИД" = "Н_ЛЮДИ"."ИД"
5 WHERE ("Н_ЛЮДИ"."ФАМИЛИЯ" < 'Афанасьев' AND "Н_ВЕДОМОСТИ"."ДАТА" = '1998-01-05'::date);

```

Индексы:

1. Для "Н_ЛЮДИ"."ИД" индекс не нужен, ибо этот атрибут является первичным ключом (индекс B-tree создаётся по умолчанию).

2. Индекс для внешнего ключа "Н_ВЕДОМОСТИ"."ЧЛВК_ИД" (B-tree) оптимизирует соединение таблиц "Н_ВЕДОМОСТИ" и "Н_ЛЮДИ".

CREATE INDEX ON "Н_ВЕДОМОСТИ" USING BTREE ("Н_ВЕДОМОСТИ"."ЧЛВК_ИД")

tablename	attname	n_distinct	null_frac	avg_width	correlation
Н_ВЕДОМОСТИ	ДАТА	3261	0	4	0.51

Уникальные значения: 3261

NULL: 0

Средний размер значений: 4 байта

В таблице значений: 5118

Уровень корреляции: 0.5

Можно сделать вывод, что данный индекс можно использовать, так как много уникальных значений, и сортировка сделана наполовину.

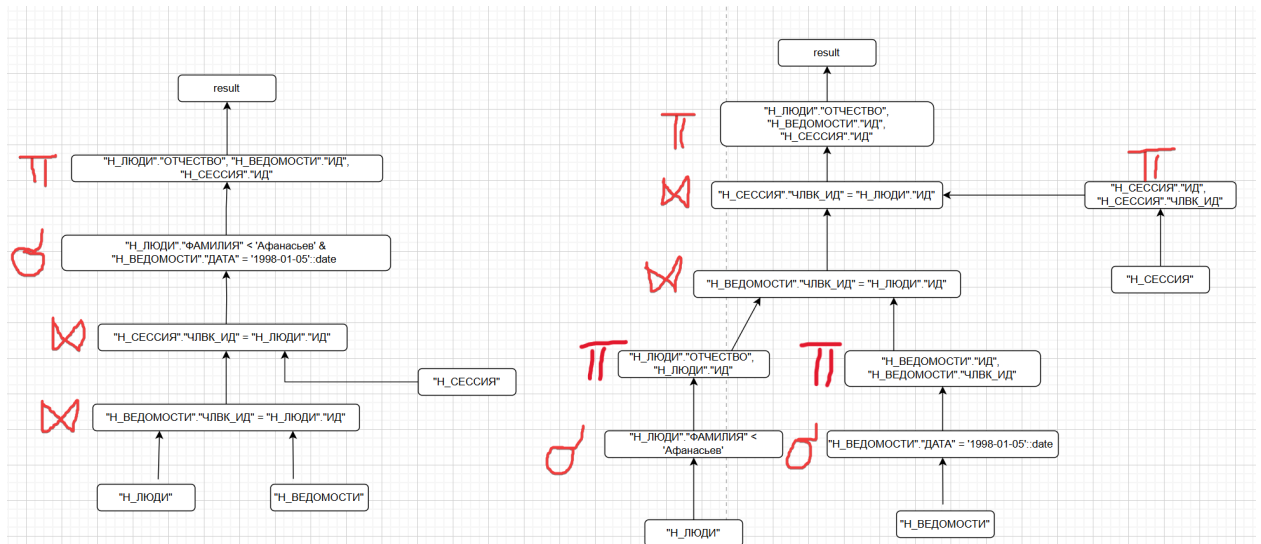
3. Индекс для м "Н_ВЕДОМОСТИ"."ДАТА" уже разобран.

4. Индекс для "Н_ЛЮДИ"."ФАМИЛИЯ" (B-tree) необходим, вследствие высокого уровня уникальных элементов (73%) и крайне низкого уровня корреляции (0.03%).

tablename	attname	n_distinct	null_frac	avg_width	correlation
Н_ВЕДОМОСТИ	ДАТА	-0.7317311	0	16	-0.031050345

Всего строк - 5118.

5. Индекс для "Н_СЕССИЯ"."ЧЛВК_ИД" не необходим, так как при 3239 строках у нас всего 180 уникальных элементов.



Второй (правый) план является оптимальным, так как выборка происходит на более ранних этапах, идет соединение только нужных атрибутов, и, как следствие, размер промежуточных данных меньше. Вместо полного сканирования будет использоваться индексный скан.

```

1 EXPLAIN ANALYSE SELECT "Н_ЛЮДИ"."ОТЧЕСТВО", "Н_ВЕДОМОСТИ"."ИД", "Н_СЕССИЯ"."ИД"
2 FROM "Н_ЛЮДИ"
3     LEFT JOIN "Н_ВЕДОМОСТИ" ON "Н_ВЕДОМОСТИ"."ЧЛВК_ИД" = "Н_ЛЮДИ"."ИД"
4     LEFT JOIN "Н_СЕССИЯ" ON "Н_СЕССИЯ"."ЧЛВК_ИД" = "Н_ЛЮДИ"."ИД"
5 WHERE ("Н_ЛЮДИ"."ФАМИЛИЯ" < 'Афанасьев' AND "Н_ВЕДОМОСТИ"."ДАТА" = '1998-01-05'::date);

```

QUERY PLAN
Hash Join (cost=292.08..4696.62 rows=215 width=28) (actual time=1.524..3.257 rows=218 loops=1)
Hash Cond: ("Н_ВЕДОМОСТИ"."ЧЛВК_ИД" = "Н_ЛЮДИ"."ИД")
-> Bitmap Heap Scan on "Н_ВЕДОМОСТИ" (cost=59.60..4442.96 rows=5072 width=8) (actual time=0.212..1.362 rows=5193 loops=1)
Recheck Cond: ("ДАТА" = '1998-01-05'::date)
Heap Blocks: exact=252
-> Bitmap Index Scan on "ВЕД_ДАТА_I" (cost=0.00..58.34 rows=5072 width=0) (actual time=0.177..0.178 rows=5193 loops=1)
Index Cond: ("ДАТА" = '1998-01-05'::date)
-> Hash (cost=229.77..229.77 rows=217 width=28) (actual time=1.281..1.283 rows=295 loops=1)
Buckets: 1024 Batches: 1 Memory Usage: 25kB
-> Hash Right Join (cost=111.39..229.77 rows=217 width=28) (actual time=0.397..1.232 rows=295 loops=1)
Hash Cond: ("Н_СЕССИЯ"."ЧЛВК_ИД" = "Н_ЛЮДИ"."ИД")
-> Seq Scan on "Н_СЕССИЯ" (cost=0.00..108.52 rows=3752 width=8) (actual time=0.006..0.346 rows=3752 loops=1)
-> Hash (cost=108.68..108.68 rows=217 width=24) (actual time=0.349..0.350 rows=216 loops=1)
Buckets: 1024 Batches: 1 Memory Usage: 21kB
-> Bitmap Heap Scan on "Н_ЛЮДИ" (cost=5.96..108.68 rows=217 width=24) (actual time=0.101..0.288 rows=216 loops=1)
Recheck Cond: (("ФАМИЛИЯ")::text < 'Афанасьев'::text)
Heap Blocks: exact=91
-> Bitmap Index Scan on "ФАМ_ЛЮД" (cost=0.00..5.91 rows=217 width=0) (actual time=0.088..0.088 rows=216 loops=1)
Index Cond: (("ФАМИЛИЯ")::text < 'Афанасьев'::text)
Planning Time: 0.607 ms
Execution Time: 3.330 ms

Порядок выполнения:

Сканирование "Н_ЛЮДИ":

Используется индекс "ФАМ_ЛЮД" для поиска всех людей с фамилией меньше "Афанасьев". Создается битовая карта страниц, эти страницы читаются из таблицы "Н_ЛЮДИ" и строки перепроверяются. Результат - 216 строк.

Heap Blocks: exact=91: было прочитано 91 блок данных из таблицы "Н_ЛЮДИ".

"Н_ЛЮДИ" хешируются (21kb).

Сканирование "Н_СЕССИЯ":

Таблица "Н_СЕССИЯ" полностью сканируется. Результат: 3752 строки.

Hash Right Join "Н_СЕССИЯ" и "Н_ЛЮДИ":

Каждая из 3752 строк "Н_СЕССИЯ" проверяется на совпадение "ЧЛВК_ИД" с ИД в хеш-таблице "Н_ЛЮДИ" (оптимизатор запросов СУБД решил, что для выполнения LEFT JOIN эффективнее будет поменять таблицы местами и выполнить RIGHT JOIN). Так как это Left Join, все строки из "Н_ЛЮДИ" попадают в результат, дополняясь данными из "Н_СЕССИЯ" при совпадении.

Результат: 295 строк.

295 строк результата предыдущего соединения загружаются в новую хеш-таблицу в памяти (25kB).

Сканирование "Н_ВЕДОМОСТИ":

Используется индекс "ВЕД_ДАТА_I" для поиска записей с датой '1998-01-05'. Создается битовая карта, читаются страницы из "Н_ВЕДОМОСТИ" и строки перепроверяются. Результат: 5193 строки.

Hash Join "Н_ВЕДОМОСТИ" и результата первого соединения:

Каждая из 5193 строк "Н_ВЕДОМОСТИ" проверяется на совпадение ЧЛВК_ИД с "Н_ЛЮДИ"."ИД" из хеш-таблицы. Результат: 218 строк.

3. Вывод

При выполнении лабораторной работы я познакомился с использованием индексов для ускорения обработки запросов в SQL, а также с планами выполнения запросов, их построением и анализом.